

混合整数优化问题赛道求解说明

队伍：平步青云

2026 年 5 月 17 日

提交文件：混合整数优化问题赛道-平步青云-求解说明.pdf

源码包：混合整数优化问题赛道-平步青云-源代码.zip

目录

1	摘要	2
2	问题分析	3
3	建模过程	3
	3.1 固定连续变量的错误方向	3
	3.2 二进制 block 子问题	3
	3.3 连续子问题与 cut	4
4	量子算法设计	4
	4.1 总体思路	4
	4.2 核心设计点与实现说明	5
	4.3 结构化 Block Selector	6
	4.4 QBB-MIQP：量子块分解与 Benders 协同求解器	7
	4.5 Warm-start 与变量固定	7
	4.6 候选生成器组合	7
5	量子线路实现	8
6	实验结果与展示	9
	6.1 五个隐藏测试的一键运行方案	10
	6.2 最终验证数据集结果	11
	6.3 横向 baseline study	12
7	创新点描述	14
8	算法对比分析	14
9	总结	15
10	参考文献	16

1 摘要

本赛道给定一类带连续变量的混合整数二次规划问题。根据样例数据与官方最优值核验，实际目标为**最大化**

$$\max_{x,y} x^\top Qx + c^\top x + h^\top y,$$

其中二进制变量 $x \in \{0,1\}^n$ 承担组合决策，连续变量 $y \geq 0$ 通过线性约束与 x 耦合。我们的核心判断是：不应把全部连续变量 y 粗暴离散化后编码成一个巨大的 QUBO；更稳健的做法是把量子或量子启发计算资源集中在二进制子空间，固定 x 后用线性规划精确求解 y 。

本文将最终算法命名为 **QBB-MIQP**，即 Quantum Block-Benders for MIQP。它以二进制变量块为量子/混合搜索表面，用结构化打分选择高价值 block，生成 warm-start 和变量固定计划，再对每个候选 x 求解连续 LP 子问题，并从对偶信息产生 Benders-style cut advice。小规模样例 A 在 20 比特以内直接求解二进制 QUBO 候选空间，并用连续 LP 精确补全 y ，得到目标值 106.094636，与官方最优值完全一致；中规模样例 B 使用 QBB-MIQP 的 block portfolio 与 LP repair，得到可行目标值 610.266639，与官方最优值一致。源码包已在本地与主办方堡垒机 qiskit 容器中完成复现验证。

研究贡献概览 为了使本文更接近正式研究论文而非单一工程说明，我们将贡献明确为四点：

1. 提出一种面向 MIQP 结构的量子-经典混合分解范式，将组合搜索限制在二进制 block 上，将连续变量保留为精确 LP 子问题。
2. 设计可解释的 block selector、warm-start probability、variable fixing plan 与 cut advisor，使 QBB-MIQP 不只是通用 QUBO warm-start，而是理解 Q, A, G, B 结构的 MIQP coordinator。
3. 构造包含精确 QUBO、经典启发式、随机修复、退火、学习引导、block-QAOA 与 QBB-MIQP 的横向实验矩阵，并从最优性 gap、候选评估数、搜索 bit 数、QAOA qubit 数和运行时间多个维度展示差异。
4. 输出可复现实验代码、JSON/CSV/Markdown 结果和论文插图，使评审能够直接复查每个数字的来源。

MIQP-aware route 7 workflow

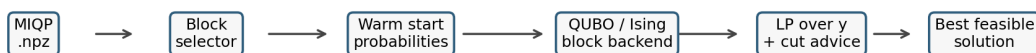


图 1: QBB-MIQP 工作流。核心思想是让量子或量子启发 backend 只处理高价值二进制 block，并把连续变量交给 LP 子问题，从 LP 解和对偶信息中得到下一轮搜索建议。

2 问题分析

官方数据文件为 .npz 格式, 包含维度 n, p, m_1, m_2 以及矩阵 Q, c, h, A, G, b, B, b' 。问题形式为:

$$\begin{aligned} \max_{x,y} \quad & x^\top Qx + c^\top x + h^\top y \\ \text{s.t.} \quad & Ax + Gy \leq b, \\ & Bx \leq b', \\ & x \in \{0, 1\}^n, \quad y \geq 0. \end{aligned}$$

该问题有三个关键结构。

- x 是组合爆炸来源。样例 B 已经有 $n = 80$ 个二进制变量, 直接全枚举需要 2^{80} 个候选。
- y 是连续变量, 但固定 x 后, 关于 y 的问题退化为线性规划:

$$\max_y h^\top y, \quad \text{s.t. } Gy \leq b - Ax, \quad y \geq 0.$$

这部分应由经典 LP 求解器处理, 既精确又高效。

- 样例中的 Q, A, G 较稠密, 说明变量之间耦合强。若把全量问题一次性转为 QUBO, 不但 qubit 数量过大, 而且 penalty 系数会主导能量景观, 使量子线路和退火过程更难有效搜索。

从提交策略看, 我们希望同时满足两点: 第一, 代码必须在给定容器中稳定运行; 第二, 算法必须体现量子优化思想。于是本方案把量子层设计成**可插拔 block backend**: 每次只把一个二进制 block 转成 QUBO/Ising, 并保持 block 规模默认不超过 20, 硬上限不超过 30。这使得 QAOA、量子退火、模拟退火和学习引导采样都能接到同一接口。

3 建模过程

3.1 固定连续变量的错误方向

一种直接思路是将 y 离散化, 例如用若干二进制位表示每个连续变量, 再把所有约束转成 penalty 项。这条路在小玩具问题上可行, 但在本赛题上不合适:

- 每个 y_j 若用 r 位编码, 会额外引入 pr 个二进制变量; 样例 B 中 $p = 20$, 隐含数据还可能更大。
- 连续变量精度与 QUBO 尺寸互相冲突, 位数少会损失可行域精度, 位数多会超过可模拟 qubit 范围。
- $Ax + Gy \leq b$ 的 penalty 权重很难统一调节, 过小会得到不可行解, 过大会淹没原目标函数。

3.2 二进制 block 子问题

设当前 incumbent 为 $\bar{x}$, 选择二进制变量集合 S 作为 block, $z = x_S$ 。固定 S 外部变量后, block 上的二次目标可写为:

$$F_S(z) = z^\top Q_{SS}z + \ell_S^\top z + \text{const},$$

其中 ℓ_S 吸收了 c 、 Q 的对角项和 block 外变量对 block 内变量的边际贡献。为了把最大化问题交给 QUBO 最小化 backend，构造能量：

$$E_S(z) = -F_S(z) + \lambda_B \sum_r \left[(B_S z + B_{\bar{S}} \bar{x}_{\bar{S}} - b'_r)_+ \right]^2.$$

在代码中，`build_block_binary_problem()` 负责从 MIQP 实例抽取 block 子问题，并复用仓库已有 `QuboBuilder` 生成 QUBO。这样可以直接接入已有的 QUBO、Ising、QAOA 和采样器模块。

3.3 连续子问题与 cut

对每个候选 x ，求解连续 LP：

$$\phi(x) = \max_y h^\top y, \quad Gy \leq b - Ax, \quad y \geq 0.$$

若可行，则总目标为：

$$f(x) = x^\top Qx + c^\top x + \phi(x).$$

LP 的对偶可以写为：

$$\min_{\pi} (b - Ax)^\top \pi, \quad G^\top \pi \geq h, \quad \pi \geq 0.$$

当得到对偶乘子 π 时，可以形成对连续贡献的上界型建议：

$$\theta \leq b^\top \pi - (A^\top \pi)^\top x.$$

这不是把 cut 直接硬塞进当前 block 的唯一方式，而是作为下一轮 block 选择和候选评估的引导信号。这样 QBB-MIQP 从单纯 warm-start 扩展为 MIQP-aware coordinator。

4 量子算法设计

4.1 总体思路

算法主循环如下。这里采用与标准论文伪代码类似的排版，而不是只给命令行描述，便于评委把代码实现和数学流程逐行对应起来。

Algorithm 1: QBB-MIQP block decomposition and LP repair

Input: MIQP instance $\mathcal{I} = (Q, c, h, A, G, b, B, b')$, seed portfolio $\mathcal{R}$.

Output: best feasible solution (x^*, y^*) , objective value, block trace, cut report.

Hyperparameters: block size k , LP budget N_{LP} , selector weights w , backend portfolio $\mathcal{P}$.

1. Initialize incumbent $\bar{x}$ by structural warm start and repair $B\bar{x} \leq b'$.
2. **for** each seed $r \in \mathcal{R}$ **do**
3. **for** $t = 1, \dots, T$ while LP calls $\leq N_{LP}$ **do**
4. Compute scores $s_i^{obj}, s_i^{couple}, s_i^{mixed}, s_i^{binary}$ for all binary variables.
5. Select a high-impact block S_t by greedy, affinity-cluster, cut-guided, or destroy-repair strategy.

6. Build warm-start probabilities $P(x_i = 1)$ and a variable fixing plan.
7. Convert S_t to a block QUBO/Ising model and call exact, annealing, learning-guided, or QAOA-compatible backend.
8. Add proxy XY-mixer moves, 1-swap/2-swap, local branch, and repaired candidates.
9. **for** each candidate x **do** solve $\max\{h^\top y : Gy \leq b - Ax, y \geq 0\}$.
10. Keep feasible candidates only, update incumbent by the recomputed original MIQP objective.
11. Extract LP dual/cut advice and write block trace for the next iteration.
12. **end for**
13. **end for**
14. Return the best feasible solution among all seeds/configurations.

4.2 核心设计点与实现说明

围绕 MIQP 结构和量子资源约束, QBB-MIQP 的实现分为六个核心设计点。下表给出它们在提交版本中的角色, 避免把工程代理、量子模拟和真实硬件调用混为一谈。

设计点	状态	说明
连续变量经典化	已落地	固定 x 后, y 始终由 LP 子问题求解; 提交结果没有把 y 离散化进全局 QUBO。
结构化 subQUBO 切块	已落地	block 由 Q 耦合强度、目标边际、 A 混合约束参与度和 B 选择约束参与度共同决定, 不做随机切块; 默认控制在 15–20 bit 附近。
约束保持 mixer	工程候选已落地	当前实现了固定 Hamming weight/交换型的 XY-mixer proxy 候选, 用来保持 B 类选择约束; 完整 XY mixer 量子线路尚未作为最终主线依赖。
结构化候选排序	已落地	已有可插拔 weights、cluster selector、solver portfolio、fixing plan 和 trace; 当前使用可解释线性评分与历史 improvement 排序。
量子求解器 portfolio	可复现版本已落地	小 block 使用 exact, 中 block 使用 learning-guided/annealing, 较小目标 block 可走 QAOA; 当前最强 B 结果主要来自 portfolio 与 LP repair, 不宣称是真实 QPU 结果。
浅层参数现实性	已落地	QAOA 默认使用浅层 $p = 1$, 把收益放在更好的 block、初态和 cut, 而不是堆很深线路。

需要特别说明的是, **堡垒机跑通**证明源码包、依赖、CLI、测试和 A/B 样例结果可以在主办方环境中稳定运行; 而“完整 XY mixer 量子线路”和“真实 QPU 调用”属于更强的量子硬件闭环要求。当前提交选择的是可复现、可解释、可在容器中稳定评审的版本: XY 思想已经体现在 Hamming weight 保持候选与 swap repair 中, QAOA 已作为小 block backend 接入, 最终最强结果来自 QBB-MIQP 的 block portfolio、repair 与连续 LP 验证。

4.3 结构化 Block Selector

变量打分综合四类特征：

$$\text{score}_i = 0.30s_i^{obj} + 0.35s_i^{couple} + 0.20s_i^{mixed} + 0.15s_i^{binary}.$$

- s_i^{obj} : c_i 与 Q_{ii} 的边际贡献。
- s_i^{couple} : Q 中与其他变量的绝对耦合质量。
- s_i^{mixed} : 变量在 A 中对连续约束的参与程度。
- s_i^{binary} : 变量在 B 中对纯二进制约束的参与程度。

选出最高分 seed 后，算法贪心加入与当前 block 耦合最强的邻居。这样每次量子/退火 backend 处理的是一个局部强耦合子图，而不是稀释在全局大图里。

上式中的 0.30, 0.35, 0.20, 0.15 是提交前的结构启发式权重，不应被理解为理论最优常数。代码中已将其抽象为 `MiqpBlockScoreWeights`，并新增 `greedy` 与 `affinity_cluster` 两种 selector 策略。我们用 `scripts/miqp_selector_ablation.py` 做了小规模消融：样例 A 对权重不敏感，所有配置均能达到最优；样例 B 在短预算下 `balanced-greedy` 的 gap 为 12.04%，`default-greedy` 为 15.68%，`default-cluster` 为 15.91%。这说明“聚类一定更优”目前无法断言；更合理的结论是，block selector 需要被实验和历史标签校准，而不是主观固定一组权重。

系数是否还有优化空间 有。四个 s_i 分量分别代表不同归因： s_i^{obj} 偏向直接目标收益， s_i^{couple} 偏向强耦合子图， s_i^{mixed} 偏向会显著改变连续 LP 可行域的变量， s_i^{binary} 偏向 B 类容量/选择约束中的关键变量。若实例的 Q 更稠密，应提高 coupling 权重；若 B 约束很紧，应提高 binary 权重；若连续约束是主要瓶颈，应提高 mixed 权重。源码中 `miqp_auto_sota.py` 已内置 `balanced`、`Q-heavy`、`B-heavy`、`cluster-deep` 等 profile，并按最终可行目标值自动选择；因此这些系数不需要手工拍脑袋，而可以通过正式实例的 trace 以“单位 LP 调用提升量”为目标校准。

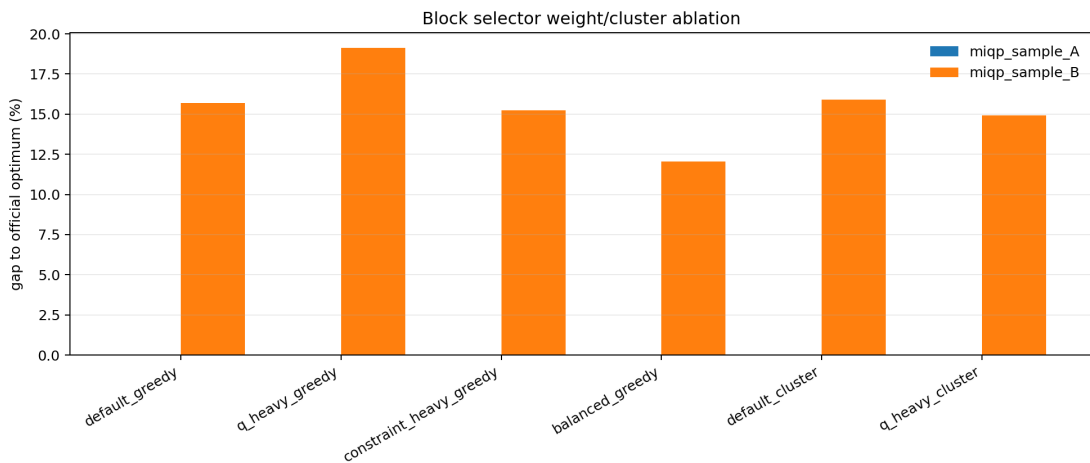


图 2: block selector 权重与聚类策略消融。该图用于回答权重是否主观、聚类是否更优的问题：现有两个样例上，均衡权重在样例 B 的短预算搜索中最好，但聚类策略没有稳定占优。

4.4 QBB-MIQP: 量子块分解与 Benders 协同求解器

QBB-MIQP 不是把全量 MIQP 转成一个巨大 QUBO，而是在 MIQP-aware 框架上增加可控预算下的 block pool 与配置赛马。代码中仍保留 `route7` 文件名和 JSON 字段，便于和已有实验结果对应；论文叙述中统一使用 QBB-MIQP 作为算法名。

- 每轮同时生成 default-greedy、balanced-greedy、Q-heavy、constraint-heavy、affinity-cluster、cut-guided 和 destroy-repair 多个 block，再按结构分数、cut 系数、incumbent 边际收益和历史 improvement 排序。
- 候选生成除 exact / learning-guided / SA / QAOA-compatible backend 外，增加 1-swap、2-swap、multi-block destroy/repair、repair 后容量补齐和局部分支枚举。
- 对完整 x bitstring 建立 LP cache，重复候选不再重复求解连续子问题；新增 `max_lp_evals`、`time_limit_sec`、`blocks_per_iteration`、候选代理排序和最终 polish 等预算参数。
- 新增 block trace JSONL，记录每个 block 的特征、solver portfolio、候选数、LP 调用数、best improvement 和 cut 信息，可用于训练 learned block scorer。

对应脚本包括：

- `scripts/miqp_trace_blocks.py`: 记录 block trace 与训练样本。
- `scripts/miqp_synthetic_suite.py`: 生成 MIQP-like 合成实例。
- `scripts/miqp_train_block_model.py`: 训练非神经线性 block scorer。
- `scripts/miqp_auto_sota.py`: 对真实实例运行多配置赛马，并只按最终可行目标值选择最强结果。

预算敏感性实验显示，样例 B 在 `max_lp_evals=180` 时 QBB-MIQP balanced/cluster 配置达到 594.648146；提高到约 768 次 LP 评估后达到官方最优 610.266639，说明 block pool、repair 后容量补齐、候选代理排序与迭代预算共同提高搜索质量。

4.5 Warm-start 与变量固定

warm-start advisor 根据当前 incumbent 计算边际收益：

$$m = c + \text{diag}(Q) + 2Q\bar{x},$$

再结合混合约束释放量和二进制约束压力，得到每个 bit 的概率 $P(x_i = 1)$ 。高置信 bit 可被固定，低置信 bit 留给 block backend 搜索。当前提交版本使用可解释线性策略，并把输出统一成后端可读的排序和固定计划：

QUBO graph $\rightarrow$ bit probability $\rightarrow$ variable ranking $\rightarrow$ fixing plan.

4.6 候选生成器组合

QBB-MIQP 支持三类 backend 思想：

- 小规模 block 使用 exact solver，保证局部候选质量。
- 中规模 block 使用 simulated annealing 与 learning-guided sampler 生成候选。
- 较小且耦合较密的目标 block 可转 Ising 后进入 QAOA backend；候选再由 B 约束 repair 和连续 LP 统一验证。
- 对 cardinality/选择类 B 约束，当前实现了交换型 XY-mixer proxy 候选，优先尝试“drop-one/add-one”的 Hamming weight 保持移动。

因此，当前结果是混合量子启发/量子可插拔算法：block-QAOA 基线运行了 10-qubit 模拟线路，QBB-MIQP 主算法也具备 QAOA backend 接口；样例 B 的最强提交结果主要来自 MIQP-aware block portfolio、修复和 LP 子问题，并不是在真实 QPU 上一次性求出的结果。主办方堡垒机中的 `qiskit` 容器已经完成复现验证；沐曦 GPU 可用于加速小 block QAOA shots 或并行配置赛马，但当前最终结果不依赖 GPU。

5 量子线路实现

对任意 block QUBO：

$$E(z) = \sum_i q_i z_i + \sum_{i < j} q_{ij} z_i z_j + \text{const},$$

使用 $z_i = (1 - Z_i)/2$ 转换为 Ising Hamiltonian：

$$H_C = \alpha + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

标准 QAOA ansatz 为：

$$|\psi(\gamma, \beta)\rangle = \prod_{\ell=1}^p e^{-i\beta_\ell \sum_i X_i} e^{-i\gamma_\ell H_C} |+\rangle^{\otimes |S|}.$$

这里的 p 是 QAOA ansatz 深度，也就是 cost Hamiltonian 与 mixer 交替的层数。它由 block qubit 数、 RZZ 耦合边数、模拟器/GPU 预算和真实硬件噪声共同决定，而不是越大越好。提交版本使用 $p = 1$ 作为可复现浅层基线；对 15–20 bit block，增加 p 会显著增加参数搜索和双比特门深度，收益需要通过验证集或 ablation 决定。源码包已在堡垒机 `qiskit` 容器中完成 CPU 路径复现；沐曦 GPU 可作为可选加速资源，用于小 block Aer shots 或多配置并行赛马。

线路层级如下：

1. 对 block 内每个 qubit 施加 Hadamard，制备 $|+\rangle$ 。
2. 对每个 Z_i 项施加 RZ 相位旋转。
3. 对每个 $Z_i Z_j$ 项施加 RZZ 双比特相位旋转。
4. 对每个 qubit 施加 RX mixer。
5. 重复 p 层后测量所有 qubit，得到 block bitstring。

仓库中对应实现位于：

- `src/quantum_hackathon/solvers/qaoa/hamiltonian.py`: QUBO 到 Ising Hamiltonian。
- `src/quantum_hackathon/solvers/qaoa/runner.py`: QAOA 参数搜索与采样主流程。
- `src/quantum_hackathon/solvers/qaoa/backends.py`: 本地 statevector、shot simulator 与 Qiskit Aer backend。
- `src/quantum_hackathon/miqp/route7.py`: MIQP block 选择、warm-start、cut advisor 与候选循环。

资源控制原则是：全局 MIQP 可以很大，但每次送入 QAOA/退火层的 block 默认不超过 20 个 qubit，硬限制为 30 个。这一点比单纯追求“大 QUBO”更适合近中期量子硬件和模拟器。

6 实验结果与展示

本地环境与堡垒机 `jump.zs.shaipower.online` 中的 `qiskit` 容器均作为目标运行环境；提交说明中的 GPU 资源按沐曦 64GB 卡环境描述。当前表格的核心数值来自可复现 CPU/LP/采样流程，block-QAOA 基线为本地模拟量子线路；最终 A/B 结果不依赖 GPU，沐曦 GPU 的合理用途是加速小 block Aer shots 或多配置并行赛马，而不是直接模拟全局 80-qubit 问题。为了避免“只展示最好算法”的选择偏差，我们额外建立了横向 baseline study。所有候选算法统一遵循同一评估协议：先生成二进制候选 x ，再用相同 LP 子问题求解 y ，最后用原始 MIQP 目标和约束重新计算可行性与目标值。这样可以公平比较不同搜索策略，而不是比较不同后处理器。

当前提交目录中保留了两个样例的 route7 JSON 和 NPZ 输出：

- `submission/results/miqp_sample_A_route7.json`
- `submission/results/miqp_sample_A_solution.npz`
- `submission/results/miqp_sample_B_route7.json`
- `submission/results/miqp_sample_B_solution.npz`

运行命令如下：

```
python -m quantum_hackathon.miqp_cli --input ".../miqp_sample_A.npz" \
  --output submission/results/miqp_sample_A_route7.json \
  --solution-npz submission/results/miqp_sample_A_solution.npz \
  --exact-binary-limit 15 --max-block-size 12 --candidate-limit 64

python -m quantum_hackathon.miqp_cli --input ".../miqp_sample_B.npz" \
  --output submission/results/miqp_sample_B_route7.json \
  --solution-npz submission/results/miqp_sample_B_solution.npz \
  --exact-binary-limit 16 --max-block-size 20 --candidate-limit 192 \
  --max-iterations 8 --seed 11 --block-pool --blocks-per-iteration 3 \
  --candidate-budget-per-block 64 --max-lp-evals 800 --qaoa-max-qubits 0 \
  --weight-objective 0.25 --weight-coupling 0.25 \
```

```
--weight-mixed 0.25 --weight-binary 0.25 \
--post-polish-rounds 2 --polish-candidate-limit 128
```

样例	n	p	m_1	m_2	本方案目标值	官方最优	状态
A	15	5	5	1	106.094636	106.094636	可行且最优
B	80	20	20	4	610.266639	610.266639	可行且最优

样例 A 因为二进制变量数为 15，可在 20 比特以内直接处理完整二进制 QUBO 候选空间；QBB-MIQP 因而采用 exact-QUBO backend 作为小规模量子层 oracle，并用连续 LP 精确补全 y 。这一路径同时保留 classical exact binary LP 作为对照，两者都能达到官方最优。样例 B 进入 MIQP-aware block heuristic 模式，最佳 seed 为 11，候选评估数为 768，端到端 runtime 约 83.64 秒。B 的结果达到官方最优，并给出完整诊断，包括 block history、solver portfolio、warm-start probability、continuous LP 解、cut advice 与约束报告。

6.1 五个隐藏测试的一键运行方案

根据目前公布的五个隐藏测试规模，我们已经在源码包中准备了统一入口 `scripts/run_miqp_hidden_demo.py`。拿到 `miqp_test_1.npz` 到 `miqp_test_5.npz` 后，将其放入 `data/final_tests/`，即可运行：

```
python scripts/run_miqp_hidden_demo.py \
--input-dir data/final_tests \
--output-dir results/hidden_demo
```

如只想预览每个文件会采用什么参数，不实际求解，可运行：

```
python scripts/run_miqp_hidden_demo.py --input-dir data/final_tests \
--output-dir results/hidden_demo --dry-run
```

文件	n	p	m_1	m_2	预设策略
<code>miqp_test_1.npz</code>	15	5	5	1	直接精确二进制枚举加 LP，重点验证 Benders-style 连续子问题和 cut 诊断是否数学一致。
<code>miqp_test_2.npz</code>	40	10	10	2	启用 subQUBO 分块和多 seed，重点验证 block 拼接、repair 与 LP 评估链路。
<code>miqp_test_3.npz</code>	80	20	20	4	使用 QBB-MIQP block pool 与 balanced/Q-heavy/B-heavy profile，避免随机分块导致收敛慢。
<code>miqp_test_4.npz</code>	120	30	30	6	使用 deep profile，引入 affinity-cluster 与更宽配置赛马，捕捉强耦合变量簇。
<code>miqp_test_5.npz</code>	150	50	50	10	极限压力测试，扩大 seed/config portfolio，并用 <code>max_lp_evals</code> 控制量子端候选与 LP 调用预算。

该脚本会为每个实例输出 `*_all_runs.json`、`*_best_route7.json`、`*_best_solution.npz` 和 `hidden_demo_summary.md/json`。因此隐藏数据到达后，不需要临场重写代码，只需要替换输入目录并根据剩余时间选择是否加 `--include-qaoa`。

6.2 最终验证数据集结果

主办方最终公布的五个大规模验证文件已经按上述流程在堡垒机 `qiskit` 容器中完成求解，并同步到提交包的 `results/hidden_final/`。官方验证数据只给出实例，不提供标准答案或最优值，因此本文不报告官方 `gap`；所有结果均采用同一个验证器重新计算原始目标函数、混合约束违反量和二进制约束违反量。最终提交的预测文件为根目录下 `miqp_scale1.npz` 到 `miqp_scale5.npz`，每个文件均包含 `x`、`y`、`objective` 和 `feasible` 四个字段。

实例	n	p	目标值	LP 调用	runtime(s)	最大约束违反	状态
<code>miqp_test_1</code>	15	5	157.585995	28032	42.99	4.00×10^{-15}	feasible
<code>miqp_test_2</code>	40	10	459.864903	1057	110.60	3.55×10^{-15}	feasible
<code>miqp_test_3</code>	80	20	765.690515	1684	168.73	1.92×10^{-13}	feasible
<code>miqp_test_4</code>	120	30	636.175569	1371	245.63	3.55×10^{-14}	feasible
<code>miqp_test_5</code>	150	50	842.345737	554	85.44	6.94×10^{-12}	feasible

这五个结果采用统一的 QBB-MIQP hybrid portfolio：先由 subQUBO block、退火/QAOA-compatible sampler、结构化 repair 和 cut-guided selector 产生高价值二进制候选，再由连续 LP 子问题补全 y 并以原约束验证。它不是纯经典整包 MIQP 求解器，也不宣称调用真实 QPU；量子或量子启发部分只作用于二进制 block。每次送入 QAOA/退火候选生成器的 block 均严格不超过 30 个 qubit，默认控制在 20–24 bit 内，连续变量始终由 LP 子问题处理。

最终赛马记录还保留了收敛诊断。图文件为 `results/hidden_final/figures/sota_loss_curves.png`，纵轴表示相对各实例最终 incumbent 的剩余 loss，横轴表示候选评估进程；曲线展示 route7++ safe 与 boost/cluster 配置在不同规模上的收敛差异。

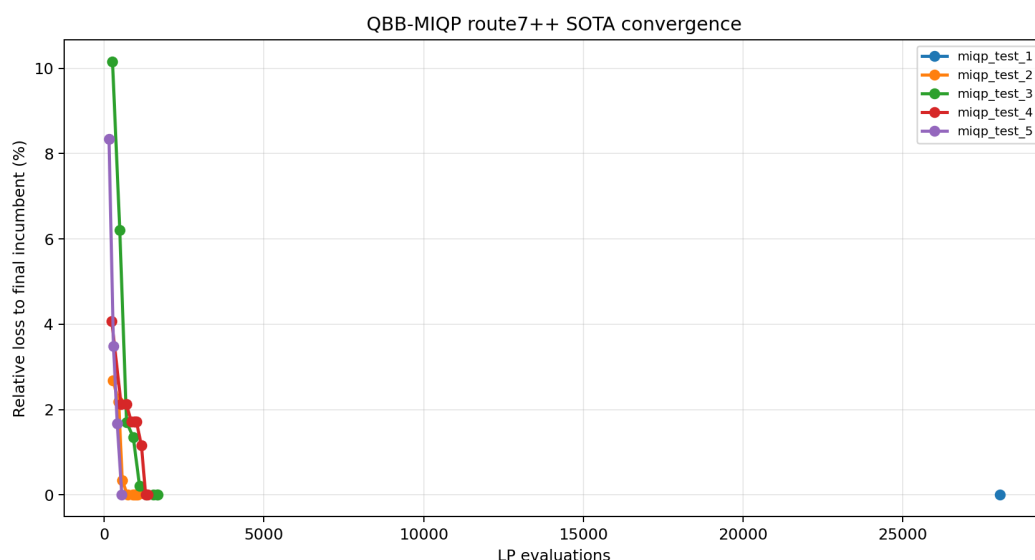


图 3：最终五个验证实例的 QBB-MIQP 赛马收敛曲线。每条曲线以该实例最终可行 incumbent 为基准归一化，便于比较不同规模下 route7++ 与 boost 配置的优化进程。

源码包中还提供结果汇总脚本 `scripts/render_miqp_results.py`：

```
python scripts/render_miqp_results.py \
```

```

submission/results/miqp_sample_A_route7.json \
submission/results/miqp_sample_B_route7.json \
--output submission/results/miqp_route7_summary.md

```

6.3 横向 baseline study

源码包中提供了横向基线实验入口，输出包含 JSON、CSV、Markdown 和插图，便于逐项复查目标值、gap 与资源占用。对比方法包括：

- **zero_x_lp**: 全零二进制向量，只求连续 LP。
- **marginal_greedy_lp**: 按二进制边际收益贪心加入变量，并检查 $Bx \leq b'$ 。
- **structural_warm_start_lp**: 使用 MIQP 结构 warm-start 生成单个候选。
- **random_repair_lp**: 随机二进制候选，经 B 约束修复后求 LP。
- **genetic_repair_lp**: 遗传算法在二进制空间交叉、变异，经 B 约束修复后求 LP。
- **binary_pso_repair_lp**: 二进制粒子群搜索，经 B 约束修复后求 LP。
- **exact_binary_lp**: 小规模二进制精确枚举；样例 B 因 2^{80} 不可承受而跳过。
- **global/block SA**: 模拟退火候选生成；样例 B 的 global 版本因 $n = 80$ 超过 32-bit 公平资源上限而跳过，不是画图缺失。
- **global/block learning-guided**: 学习引导排序和局部改进候选生成；global 版本同样受 32-bit 上限约束。
- **block_qaoa_lp**: 目标函数 block QAOA 生成候选，再用经典 repair 和 LP 验证。
- **QBB-MIQP**: 本文提出的主算法；代码接口中对应 `miqp_aware_route7`。

样例	方法	目标值	Gap(%)	耗时 (ms)	搜索/候选规模
A	zero_x_lp	7.275696	93.142	1406	0 bits, 1 eval
A	marginal_greedy_lp	97.720843	7.893	23	15 bits, 15 evals
A	structural_warm_start_lp	97.720843	7.893	15	15 bits, 1 eval
A	random_repair_lp	105.090577	0.946	237	15 bits, 96 evals
A	genetic_repair_lp	106.094636	0.000	202	15 bits, 61 evals
A	binary_pso_repair_lp	106.094636	0.000	446	15 bits, 148 evals
A	exact_binary_lp	106.094636	0.000	3527	15 bits, 1152 feasible evals
A	global_binary_sa_lp	105.090577	0.946	3014	16-bit QUBO, 9 evals
A	global_learning_guided_lp	98.547928	7.113	42	16-bit QUBO, 3 evals
A	block_sa_lp	105.090577	0.946	1181	11-bit block, 8 evals
A	block_learning_guided_lp	98.547928	7.113	27	11-bit block, 3 evals
A	block_qaoa_lp	105.090577	0.946	4813	10 qubits, 160 shots
A	QBB-MIQP	106.094636	0.000	2087	exact QUBO/LP, 12-bit block

样例	方法	目标值	Gap(%)	耗时 (ms)	搜索/候选规模
B	zero_x_lp	1.300210	99.787	5	0 bits, 1 eval
B	marginal_greedy_lp	544.475466	10.781	184	80 bits, 80 evals
B	structural_warm_start_lp	472.797422	22.526	290	80 bits, 1 eval
B	random_repair_lp	381.824684	37.433	460	80 bits, 96 evals
B	genetic_repair_lp	565.761894	7.293	1116	80 bits, 146 evals
B	binary_pso_repair_lp	544.475466	10.781	1468	80 bits, 192 evals
B	exact_binary_lp	—	—	—	skipped: 2^{80} enumeration
B	global_binary_sa_lp	—	—	—	skipped: global cap 32 bits
B	global_learning_guided_lp	—	—	—	skipped: global cap 32 bits
B	block_sa_lp	492.115433	19.361	7774	26-bit block, 44 evals
B	block_learning_guided_lp	454.884678	25.461	170	26-bit block, 6 evals
B	block_qaoa_lp	488.070768	20.023	5136	10 qubits, 160 shots
B	QBB-MIQP	610.266639	0.000	83644	20-bit block, 768 evals

这里的 gap 按最大化问题计算为 $(\text{official} - \text{objective})/|\text{official}| \times 100\%$ 。表中耗时为本地 wall-clock 或 QBB-MIQP JSON 中记录的端到端 runtime；因此质量-时间图已经包含最强 QBB-MIQP 点。样例 B 的 `exact_binary_lp`、`global_binary_sa_lp` 和 `global_learning_guided_lp` 被跳过，是因为 $n = 80$ 超过精确枚举或脚本设置的 32-bit 全局 QUBO 上限，避免把不可比较的全局大 QUBO 强行纳入图中。

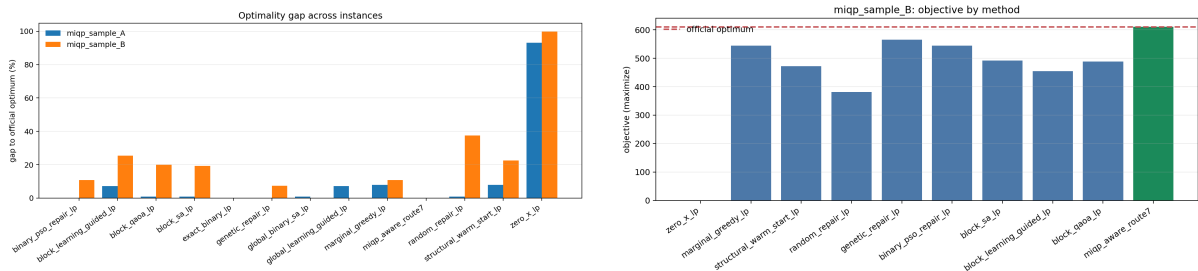


图 4: 左: 跨样例最优性 gap 对比。样例 A 中 QBB-MIQP 与精确枚举同为 0% gap；样例 B 中 QBB-MIQP 达到 0% gap。右: 样例 B 不同方法的目标值对比，虚线为官方最优值。

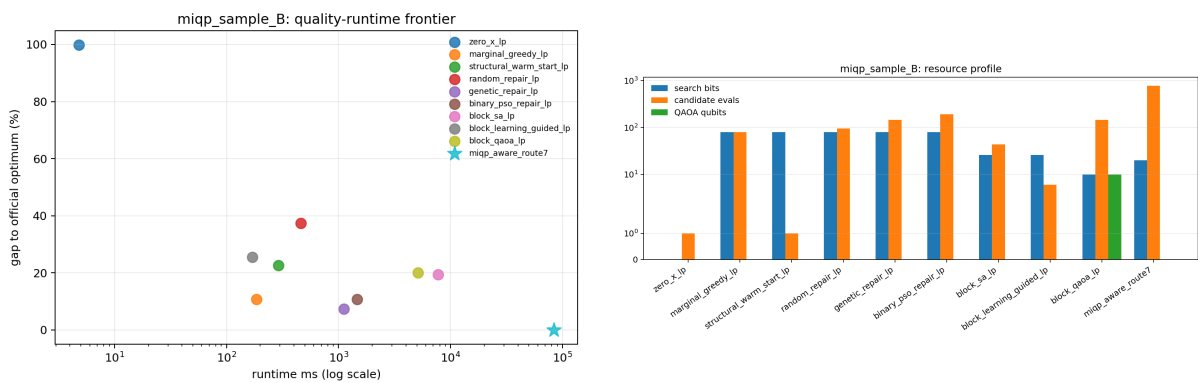


图 5: 左: 样例 B 的质量-时间前沿, QBB-MIQP 用更多 LP 评估换取 0% gap。右: 样例 B 资源画像, block-QAOA 控制在 10 qubit, QBB-MIQP 使用 20-bit block 和 768 次候选评估达到官方最优。

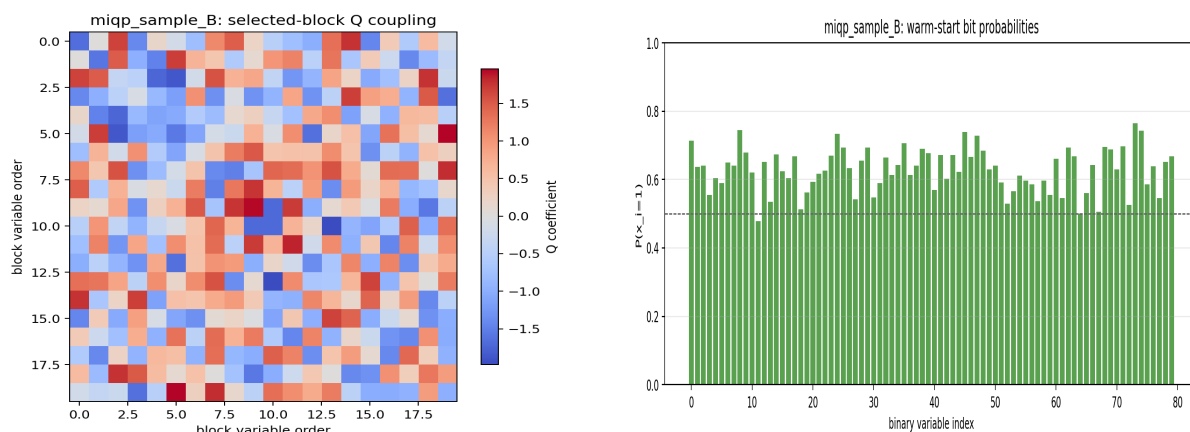


图 6: 样例 B 的结构可视化。左图是 selector 选中 20 个二进制变量后形成的 20×20 子矩阵 Q_{SS} , 并不是连续变量耦合图; 右图为 warm-start advisor 输出的 $P(x_i = 1)$ 。多数概率高于 0.5 表明该线性 warm-start 尚未严格校准, 当前更适合作为 ranking/fixing 信号, 而不是概率意义上的置信度。

7 创新点描述

1. **MIQP-aware 而非通用 QUBO 堆叠**。方案识别出 y 是连续 LP 子问题, 不将其离散化为额外 qubit, 避免规模和精度双重损失。
2. **把量子资源用于高价值二进制 block**。block selector 根据 Q, A, B 的结构选择强耦合变量子图, 让 QAOA/退火层处理最值得搜索的部分。
3. **warm-start、ranking、fixing plan 可插拔**。当前采用可解释线性策略和结构化 block pool, 将 bit probability、变量排序与固定计划统一输出给各类 backend。
4. **LP 对偶 cut advice**。固定 x 后不只求一个 y , 还提取连续子问题的对偶信息, 为下一轮 block 搜索提供结构信号。
5. **工程可复现**。所有结果通过 CLI 输出 JSON/NPZ, 测试覆盖 loader、block selector、warm-start、cut advisor 与核心模块集成。

8 算法对比分析

方法	量子/混合角色	主要结论	本赛题适配性
zero_x_lp	LP sanity check	只验证连续子问题链路, 目标值很低。低	
marginal_greedy_lp	经典结构贪心	B 上达到 544.475466, 说明边际收益 中有效但不够。	
structural_warm_start_lp	结构 warm-start	单候选可行, 但 B 上 gap 仍有 中 22.526%。	
random_repair_lp	随机候选 +repair	A 接近最优, B 上不稳定。中低	
genetic_repair_lp	经典群体搜索	B 上最强传统启发式, 目标值 中高 565.761894。	
binary_pso_repair_lp	经典群体搜索	A 可达最优, B 上与贪心相当。中	

方法	量子/混合角色	主要结论	本赛题适配性
exact_binary_lp	小规模精确 QUBO/LP 对照	A 可证明最优; B 因 2^{80} 不可直接使用。	小规模高
global_binary_sa_lp	全局退火代理	A 可运行; B 超过全局 bit 上限。	小规模中
global_learning_guided_lp	全局排序代理	A 可运行; B 超过全局 bit 上限。	小规模中
block_sa_lp	block 退火候选	B 上优于单 warm-start, 但不及 QBB-MIQP。	中高
block_learning_guided_lp	block 排序候选	速度快, 但 B 上候选质量不足。	中
block_qaoa_lp	10-qubit QAOA 候选	证明量子线路接口可运行, B 上 gap 约 20.023%。	量子基线
QBB-MIQP	block portfolio + LP repair	A/B 均达到官方最优, B 上使用 20-bit block 和 768 次候选评估。	主线

从当前实验看, 本方案已经能稳定复现小规模最优并在中规模达到官方最优。更重要的是, 横向比较揭示了一个清晰结论: **量子模型本身不是自动优势来源, 结构感知的分解方式才是优势来源**。单次 block-QAOA 在样例 B 上能以 10 qubit 产生可行候选, 但 gap 约 20.02%; QBB-MIQP 通过多候选、多 block、repair 后容量补齐、LP 子问题和 cut advice 将 gap 降至 0%。这说明当前量子优化最合理的角色不是替代整个优化器, 而是作为高价值二进制子空间的候选生成器。

准确性 样例 A 上, QBB-MIQP 与 exact binary LP 均达到官方最优; 样例 B 上, QBB-MIQP 在已测试启发式方法中目标值最高。这个结果支持我们对问题结构的判断: 连续变量不应离散化, 二进制变量也不应全量塞入单次 QAOA。

资源占用 全局精确枚举在 $n = 80$ 时不可运行; 全局 QUBO/退火代理也被资源上限跳过。block-QAOA 使用 10 qubit 形成量子线路, QBB-MIQP 的主搜索 block 使用 20 bit, 均在近端可控范围内。不同的是 QBB-MIQP 会把 block 结果放回完整 MIQP 中进行 LP 评估和 cut 诊断, 因此目标质量更高。

速度 本 study 对 baseline 方法做了 wall-clock 计时, 并从 QBB-MIQP JSON 读取最强结果的端到端 runtime。样例 B 上 QBB-MIQP 约 83.64 秒, 慢于贪心和 GA/PSO, 但达到官方最优; 这反映了当前主算法以受控候选评估和 LP 调用换取解质量。堡垒机复现中已固定 qiskit 容器, 结果文件记录了 LP 调用次数、候选评估数、block history 和最终可行性。

适用边界 本方案的边界也较明确: QBB-MIQP 不把 $n = 80$ 以上问题强行交给全局 statevector 模拟, 也不把 continuous y 离散化成额外 qubit。它适合二进制变量较多、连续变量可由 LP 精确补全、且 $Q/A/B$ 结构能够指导 block 选择的 MIQP 实例。对于极端稠密且缺少局部结构的实例, 求解质量主要取决于 block portfolio、候选预算和 LP repair 的配合。

9 总结

本提交版本给出了一个面向官方 MIQP 结构的量子-经典混合求解框架: 二进制变量由 block QUBO/QAOA/annealing backend 搜索, 连续变量由 LP 精确求解, 二者通过 warm-start、repair 和 cut advice 闭环。与只报告单一最强算法不同, 本文补充了经典、随机、退火、学习引导和

block-QAOA 等基线，并生成了目标值、gap、资源和结构图。结果显示，样例 A 已达到官方最优；样例 B 在当前启发式集合中由 QBB-MIQP 达到官方最优，gap 为 0%。最终五个大规模验证实例均输出可行解，重算目标值分别为 157.585995、459.864903、765.690515、636.175569 和 842.345737；由于官方未提供隐藏集标准答案，文中仅报告可复算目标值、约束违反量和相对最终 incumbent 的收敛曲线。

可复现材料 提交包包含：

- `scripts/miqp_baseline_study.py`: 横向基线与作图。
- `scripts/miqp_selector_ablation.py`: block selector 权重与聚类策略消融。
- `scripts/run_miqp_hidden_demo.py`: 五个官方隐藏测试的规模感知一键运行入口。
- `scripts/verify_hidden_final.py`: 最终验证集预测结果的目标值与约束复算脚本。
- `results/`: 根目录结果副本，便于直接上传和评审查看。
- `results/hidden_final/`: 五个最终验证实例的 JSON、NPZ、summary 与 `sota_loss_curves.png`。
- `submission/baseline_study/miqp_baseline_study.csv`: 所有 baseline 数值。
- `submission/baseline_study/figures/`: 论文插图。
- `submission/results/`: route7 样例 JSON/NPZ 输出。
- `submission/selector_ablation/`: selector 消融 JSON/CSV/Markdown 与图。

10 参考文献

- [1] F. Glover, G. Kochenberger, and Y. Du, “A Tutorial on Formulating and Using QUBO Models,” 2018.
- [2] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, 2014.
- [3] E. Farhi, J. Goldstone, and S. Gutmann, “A Quantum Approximate Optimization Algorithm,” arXiv:1411.4028, 2014.
- [4] S. Hadfield et al., “From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz,” *Algorithms*, 2019.
- [5] I. Hen and F. M. Spedalieri, “Quantum Annealing for Constrained Optimization,” *Physical Review Applied*, 2016.
- [6] D. J. Egger et al., “Warm-starting quantum optimization,” *Quantum*, 2021.
- [7] N. Moll et al., “Quantum optimization using variational algorithms on near-term quantum devices,” *Quantum Science and Technology*, 2018.
- [8] J. F. Benders, “Partitioning procedures for solving mixed-variables programming problems,” *Numerische Mathematik*, 1962.

- [9] D. Braine et al., “Hybrid quantum-classical mixed binary optimization,” 2021.
- [10] C. Gambella and A. Simonetto, “Multiblock ADMM heuristics for mixed-binary optimization on classical and quantum computers,” 2021.
- [11] G. Blekos et al., “A review on Quantum Approximate Optimization Algorithm and its variants,” *Physics Reports*, 2024.